PRAXISHANDBUCH

Hashcat

Das Praxishandbuch

GPU-beschleunigtes Passwort-Cracking verstehen – von der Maske bis zum eigenen Benchmark



DIESELBE FRAGE, MEHR PARALLELITÄT. NUR MIT EIGENER BERECHTIGUNG.

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

Hashcat
Das Praxishandbuch

GPU-beschleunigtes Passwort-Cracking verstehen – von der Maske bis zum eigenen Benchmark

Von Angriffsmodi über Masken und Regeln bis zu Benchmarks, Workload-Profilen und hashcat-utils.
Mit geführten Übungen mit selbst erzeugten Testhashes, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Für wen ist dieses Handbuch? 5

Aufbau 6

Kapitel 1: Was ist Hashcat? 7

1.1 Einordnung: von Grund auf GPU-zentriert 7

1.2 Typische Einsatzgebiete 7

1.3 Der grundsätzliche Ablauf 8

1.4 Dual-Use wie jedes Werkzeug dieser Reihe 8

Zusammenfassung 8

Kapitel 2: Grundlagen – GPU- statt CPU-zentriertes Cracking 9

2.1 Warum GPUs so viel schneller rechnen können 9

2.2 OpenCL und CUDA 9

2.3 Warum langsame Hash-Verfahren den Vorteil relativieren 10

Zusammenfassung 10

Kapitel 3: Installation und Laborumgebung 11

3.1 Installation 11

3.2 GPU-Treiber oder CPU-Modus 11

3.3 Eigene Testhashes erzeugen 12

3.4 Laborumgebung 12

Zusammenfassung 12

Kapitel 4: Angriffsmodi 13

4.1 Die Angriffsmodi im Überblick 13

4.2 Dictionary-Angriff (-a 0) 13

4.3 Combination-Angriff (-a 1) 13

4.4 Brute-Force- bzw. Maskenangriff (-a 3) 14

4.5 Hybrid-Angriff (-a 6 / -a 7) 14

Zusammenfassung 14

Kapitel 5: Hash-Modi erkennen und angeben 15

5.1 Warum die exakte Nummer zählt 15

5.2 Häufige Hash-Modi 15

5.3 Beispiel-Hashes zum Nachschlagen 16

5.4 Testhashes mit passendem Modus erzeugen . . . 16
Zusammenfassung . . . 16

Kapitel 6: Regeln in Hashcat . . . 17

6.1 Dasselbe Prinzip wie bei John the Ripper, andere Syntax . . . 17
6.2 Regelsyntax in Grundzügen . . . 17
6.3 Mehrere Regeldateien kombinieren . . . 18
Zusammenfassung . . . 18

Kapitel 7: Masken und eigene Zeichensätze im Detail . . . 19

7.1 Standard-Zeichensatzplatzhalter . . . 19
7.2 Eigene Zeichensätze . . . 19
7.3 Suchraum abschätzen und begrenzen . . . 19
7.4 Inkrementelle Maskenlänge . . . 20
Zusammenfassung . . . 20

Kapitel 8: Performance, Workload-Profile, Sessions und Potfile . . . 21

8.1 Benchmark . . . 21
8.2 Workload-Profile . . . 21
8.3 Sessions: lange Läufe unterbrechen und fortsetzen . . . 22
8.4 Die Potfile . . . 22
Zusammenfassung . . . 22

Kapitel 9: hashcat-utils und ergänzende Werkzeuge . . . 23

9.1 Was ist hashcat-utils? . . . 23
9.2 Wortlisten bereinigen und kombinieren . . . 23
9.3 Wörter aus bestehenden Treffern ableiten . . . 23
9.4 Wann lohnt sich ein Zusatzwerkzeug? . . . 24
Zusammenfassung . . . 24

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 25

10.1 Laboraufbau . . . 25
10.2 Checkliste: sicherer Umgang mit Testdaten . . . 25
10.3 Übungen . . . 25
Zusammenfassung . . . 27

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen . . . 28

11.1 Häufige Probleme . . . 28
11.2 Spickzettel . . . 29

11.3 Glossar 29
11.4 Weiterführende Ressourcen 30
Zusammenfassung 30

Verzeichnis der Befehlsbeispiele 31

Index 32

Vorwort

`Hashcat` ist das GPU-zentrierte Gegenstück zu `John the Ripper` aus dieser Reihe: dieselbe Grundaufgabe – Passwort-Hashes offline gegen Wortlisten, Regeln und systematische Kombinationen prüfen –, aber von Grund auf darauf ausgelegt, die massive Parallelität moderner Grafikkarten auszunutzen. Wo John the Ripper historisch bei CPU-basiertem Cracking und klassischen Unix-Formaten stark ist, zeigt Hashcat seine Stärke bei großen Hash-Mengen und rechenintensiven, aber GPU-geeigneten Verfahren.

Dieses Handbuch setzt das John-the-Ripper-Handbuch dieser Reihe direkt fort: Grundbegriffe wie Hashing, Salting und das Prinzip des systematischen Ausprobierens werden hier vorausgesetzt (John-the-Ripper-Handbuch, Kapitel 2) und nicht erneut erklärt. Stattdessen stehen die Punkte im Mittelpunkt, in denen sich Hashcat deutlich unterscheidet: Angriffsmodi, Maskensyntax, GPU-Performance und Workload-Profile.

> **ILLEGAL – niemals ohne Auftrag**
>
> Wie in den anderen Büchern dieser Reihe gilt ohne Ausnahme: Hashcat darf **ausschließlich** gegen Passwort-Hashes eingesetzt werden, zu deren Prüfung du ausdrücklich berechtigt bist – aus deinen eigenen Testsystemen, aus einem schriftlich autorisierten Auftrag, oder aus frei verfügbaren Übungsdateien, die ausdrücklich zu Lernzwecken bereitgestellt werden. Das unbefugte Beschaffen oder Knacken fremder Zugangsdaten ist nach § 202a StGB (Ausspähen von Daten) strafbar, unabhängig davon, ob der Versuch erfolgreich ist.

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- bereits das John-the-Ripper-Handbuch kennst und das GPU-zentrierte Gegenstück dazu lernen willst,
- Zugriff auf eine dedizierte Grafikkarte hast und wissen möchtest, wie viel Unterschied das praktisch macht,
- größere Hash-Mengen aus einem autorisierten Audit systematisch prüfen musst,
- dich auf Zertifizierungen vorbereitest, in denen Passwort-Audits eine Rolle spielen (eJPT, OSCP, CompTIA Security+/PenTest+).

Aufbau

Kapitel	Thema
1	Was ist Hashcat? Einordnung, Geschichte, Abgrenzung zu John the Ripper
2	Grundlagen: GPU- statt CPU-zentriertes Cracking
3	Installation und Laborumgebung
4	Angriffsmodi: Dictionary, Combination, Brute-Force, Hybrid
5	Hash-Modi erkennen und angeben
6	Regeln in Hashcat
7	Masken und eigene Zeichensätze im Detail
8	Performance: Benchmark, Workload-Profile, Sessions, Potfile
9	hashcat-utils und ergänzende Werkzeuge
10	Praxisteil: Laboraufbau und geführte Übungen
11	Troubleshooting, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Kennst du bereits das John-the-Ripper-Handbuch, wirst du in Kapitel 4 bis 7 viele vertraute Konzepte mit neuer, expliziterer Syntax wiederfinden – Hashcat benennt Angriffsmodi und Maskenplatzhalter sehr ähnlich, aber nicht identisch.

Kapitel 1: Was ist Hashcat?

Lernziele

- Du kannst Hashcat von John the Ripper funktional abgrenzen.
- Du kennst die Geschichte und die wichtigsten Einsatzgebiete von Hashcat.
- Du verstehst den grundsätzlichen Ablauf eines Hashcat-Laufs.

1.1 Einordnung: von Grund auf GPU-zentriert

John the Ripper entstand 1996 für CPU-basiertes Cracking klassischer Unix-Passwörter (John-the-Ripper-Handbuch, Kapitel 1.2). Hashcat wurde dagegen von Anfang an darauf ausgelegt, die **massive Parallelität von Grafikkarten** auszunutzen: Eine moderne GPU kann tausende Hash-Berechnungen gleichzeitig durchführen, wo eine CPU nur eine Handvoll Kerne zur Verfügung hat.

Merkmal	John the Ripper	Hashcat
Historischer Ursprung	CPU, Unix-Passwortformate	GPU-Parallelisierung von Anfang an
Rechenweg	CPU (optional GPU-Erweiterungen)	GPU über OpenCL/CUDA (optional reiner CPU-Modus)
Angriffsmodi	Modi mit Namen (Single, Wordlist, Incremental)	durchnummerierte Modi (`-a 0` bis `-a 7`)
Typische Stärke	Unix-Systemdateien, CPU-only-Umgebungen	große Hash-Mengen, GPU-geeignete Formate

Gut zu wissen

Hashcat wurde von Jens Steube (bekannt als „atom") entwickelt und hat sich seit den frühen 2010er-Jahren zum De-facto-Standard für GPU-beschleunigtes Passwort-Cracking entwickelt. Die heute verbreitete Version vereint das frühere „oclHashcat" (GPU) und das klassische „Hashcat" (CPU) in einem einzigen Programm.

1.2 Typische Einsatzgebiete

- **Passwort-Audits mit großen Hash-Mengen**, bei denen die GPU-Beschleunigung einen Zeitunterschied von Stunden zu Minuten ausmachen kann.
- **Prüfung GPU-geeigneter Hash-Verfahren** wie NTLM, einfaches SHA-256 oder WPA/WPA2-Handshakes.
- **Post-Exploitation in autorisierten Pentests**, analog zum Einsatz von John the Ripper (Ethical-Hacking-Handbuch, Kapitel 7).
- **Benchmark-Vergleiche**, um die reale Auswirkung unterschiedlicher Hash-Verfahren auf die Sicherheit einer Anwendung greifbar zu machen (Kapitel 8).

1.3 Der grundsätzliche Ablauf

Beispiel 1.1: Typischer Ablauf eines Hashcat-Laufs (Konzept)

```
1. Hashes beschaffen (nur aus autorisierter Quelle)
2. Hash-Modus (-m) bestimmen (Kapitel 5)
3. Angriffsmodus (-a) wählen: Dictionary, Combination, Brute-Force, Hybrid (Kapitel 4)
4. Optional Regeln oder Masken einsetzen (Kapitel 6-7)
5. Hashcat laufen lassen, Fortschritt und Status beobachten (Kapitel 8)
6. Gefundene Passwörter aus der Potfile einsehen und dokumentieren
```

1.4 Dual-Use wie jedes Werkzeug dieser Reihe

LEGAL – nur mit Auftrag/Einwilligung

Auch Hashcat hat keine Funktion, die „nur für legale Zwecke" reserviert ist. Der einzige Unterschied zwischen einem legitimen Passwort-Audit und einem Angriff ist – wie im gesamten Ethical-Hacking-Handbuch – die **Berechtigung**: eigene Testsysteme, ein schriftlicher Auftrag mit definiertem Scope, oder ausdrücklich zu Übungszwecken bereitgestellte Hash-Dateien (Kapitel 3 und 10).

Zusammenfassung

Zusammenfassung

- Hashcat ist von Grund auf für GPU-Parallelisierung ausgelegt, John the Ripper historisch für CPU-basiertes Cracking.
- Beide durchnummerieren bzw. benennen ihre Angriffsmodi unterschiedlich, lösen aber dieselbe Grundaufgabe.
- Haupteinsatzgebiete sind große Hash-Mengen, GPU-geeignete Verfahren, Post-Exploitation und Benchmark-Vergleiche.
- Wie jedes Werkzeug dieser Reihe entscheidet ausschließlich die Berechtigung über Legalität, nicht die Technik.

Kapitel 2: Grundlagen – GPU- statt CPU-zentriertes Cracking

Lernziele

- Du verstehst, warum GPUs für bestimmte Hash-Verfahren so viel schneller sind als CPUs.
- Du kennst den Unterschied zwischen OpenCL und CUDA als Rechenwege.
- Du verstehst, warum absichtlich langsame Hash-Verfahren den GPU-Vorteil wieder einschränken.

2.1 Warum GPUs so viel schneller rechnen können

Eine **CPU** hat wenige, aber sehr vielseitige Rechenkerne, optimiert für komplexe, sequenzielle Aufgaben. Eine **GPU** hat dagegen tausende einfachere Kerne, die ideal für die immer gleiche, massenhaft parallel wiederholte Berechnung geeignet sind – genau das Muster, das Passwort-Cracking braucht: denselben Hash-Algorithmus millionenfach auf unterschiedliche Kandidaten anwenden.

Beispiel 2.1: Grobe Größenordnung (stark vereinfacht, Konzept)

```
CPU:  wenige bis einige Dutzend Kerne, je sehr leistungsfähig
GPU:  tausende Kerne, je einfacher, aber massiv parallel einsetzbar
-> Für "dieselbe Rechnung millionenfach wiederholen" liegt der Vorteil klar bei der GPU
```

Gut zu wissen

Dieser Vorteil gilt nicht für jede Aufgabe gleichermaßen: Verfahren, die absichtlich viel Arbeitsspeicher pro Berechnung benötigen (z. B. **scrypt** oder **Argon2**), lassen sich auf GPUs deutlich weniger effizient parallelisieren als rechenintensive, aber speicherarme Verfahren. Das ist kein Zufall, sondern eine bewusste Designentscheidung dieser moderneren Hash-Verfahren.

2.2 OpenCL und CUDA

Hashcat spricht Grafikkarten über eine von zwei Schnittstellen an:

Schnittstelle	Hersteller-Bindung	Typische Nutzung
OpenCL	herstellerunabhängig	AMD-, Intel- und manche NVIDIA-Karten
CUDA	NVIDIA-spezifisch	NVIDIA-Karten, oft mit zusätzlicher Geschwindigkeit gegenüber OpenCL auf derselben Hardware

Beispiel 2.2: Verfügbare Rechengeräte auflisten

```
$ hashcat -I
OpenCL Info:
  Platform ID #1: NVIDIA Corporation
    Backend Device ID #1: NVIDIA GeForce RTX ..., skipped
CUDA Info:
  Backend Device ID #1: NVIDIA GeForce RTX ...
```

WARNUNG – Sicherheitsrisiko

Ohne passende, aktuelle Grafikkartentreiber (und bei NVIDIA zusätzlich das CUDA-Toolkit) erkennt Hashcat die GPU möglicherweise gar nicht oder fällt auf einen deutlich langsameren CPU-Modus zurück. Kapitel 3 geht auf die Einrichtung ein.

2.3 Warum langsame Hash-Verfahren den Vorteil relativieren

Der GPU-Vorteil ist am größten bei schnellen, einfachen Hash-Verfahren (z. B. unsalted MD5 oder NTLM). Bei absichtlich langsamen Verfahren wie bcrypt mit hohem Kostenfaktor schrumpft der Geschwindigkeitsunterschied zur CPU erheblich, weil jede einzelne Berechnung selbst schon aufwendig ist – es gibt dann schlicht weniger zu parallelisieren.

Beispiel 2.3: Relativer GPU-Vorteil nach Hash-Verfahren (Konzept, keine exakten Werte)

```
unsalted MD5:    GPU sehr deutlich schneller als CPU
gesaltetes SHA-256: GPU deutlich schneller als CPU
bcrypt (hoher Kostenfaktor): GPU-Vorteil gegenüber CPU deutlich kleiner
```

Das ist derselbe Grundgedanke wie im John-the-Ripper-Handbuch (Kapitel 2.2): Nicht die Wahl des Werkzeugs, sondern das verwendete Hash-Verfahren bestimmt am Ende, wie realistisch ein Cracking-Versuch überhaupt ist.

Zusammenfassung

Zusammenfassung

- GPUs rechnen bei massenhaft paralleler, gleichartiger Arbeit (wie Passwort-Cracking) deutlich schneller als CPUs.
- Hashcat spricht Grafikkarten über OpenCL (herstellerunabhängig) oder CUDA (NVIDIA-spezifisch) an.
- Speicherintensive Verfahren wie scrypt und Argon2 lassen sich schlechter parallelisieren als einfache, schnelle Hashes.
- Absichtlich langsame Verfahren wie bcrypt relativieren den GPU-Vorteil gegenüber der CPU erheblich.

Kapitel 3: Installation und Laborumgebung

Lernziele

- Du installierst Hashcat unter Kali, Debian/Ubuntu und Fedora/Arch.
- Du richtest die nötigen GPU-Treiber ein oder nutzt gezielt den CPU-Modus.
- Du erzeugst dir eigene Testhashes für alle folgenden Übungen.

3.1 Installation

Beispiel 3.1: Hashcat installieren

```
# Kali Linux: bereits vorinstalliert
$ hashcat --version

# Debian/Ubuntu
$ sudo apt update && sudo apt install hashcat

# Fedora
$ sudo dnf install hashcat
# Arch Linux
$ sudo pacman -S hashcat
```

3.2 GPU-Treiber oder CPU-Modus

Beispiel 3.2: Prüfen, welche Rechengeräte Hashcat erkennt

```
$ hashcat -I
```

Situation	Vorgehen
Dedizierte NVIDIA-GPU vorhanden	aktuellen proprietären Treiber und das CUDA-Toolkit installieren
Dedizierte AMD-GPU vorhanden	aktuellen Treiber mit OpenCL-Unterstützung installieren
Nur integrierte Grafik oder virtuelle Maschine ohne GPU-Durchreichung	Hashcat läuft auch im reinen CPU-Modus, deutlich langsamer

Gut zu wissen

In einer virtuellen Maschine ist eine echte GPU-Durchreichung (Passthrough) oft nicht ohne Weiteres möglich. Für die Übungen in diesem Buch reicht der **CPU-Modus** völlig aus – die Konzepte (Angriffsmodi, Masken, Regeln) sind identisch, nur spürbar langsamer als mit echter GPU-Beschleunigung.

Beispiel 3.3: Hashcat im reinen CPU-Modus erzwingen

```
$ hashcat -D 1 -a 0 -m 0 testhashes.txt testliste.txt
```

`-D 1` erzwingt die Nutzung von CPU-Rechengeräten; `-D 2` steht für GPU-Geräte. Ohne diese Option nutzt Hashcat automatisch alle erkannten, geeigneten Geräte.

3.3 Eigene Testhashes erzeugen

Beispiel 3.4: Testhashes aus frei erfundenen Passwörtern erzeugen

```
$ echo -n "testpass123" | md5sum
$ echo -n "testpass123" | sha256sum
```

TIPP – Profi-Trick

Wie im John-the-Ripper-Handbuch gilt auch hier: Lege dir eine eigene Datei mit mehreren selbst erzeugten Hashes aus **frei erfundenen** Testpasswörtern an. Für kein einziges Beispiel in diesem Buch sind echte Zugangsdaten nötig.

3.4 Laborumgebung

Wie beim John-the-Ripper-Handbuch genügt für dieses Buch eine einzelne Kali-VM: Hashcat arbeitet ausschließlich mit lokal vorliegenden Hash-Dateien, ohne Netzwerkkomponente.

LEGAL – nur mit Auftrag/Einwilligung

Nutze ausschließlich selbst erzeugte Testhashes oder ausdrücklich zu Übungszwecken bereitgestellte Dateien (Kapitel 1.4 und 10).

Zusammenfassung

Zusammenfassung

- Hashcat ist unter Kali vorinstalliert, auf anderen Distributionen leicht nachzuinstallieren.
- `hashcat -I` zeigt, welche Rechengeräte (CPU/GPU) erkannt werden; `-D 1`/`-D 2` erzwingt gezielt CPU oder GPU.
- Ohne echte GPU-Durchreichung reicht für dieses Buch der CPU-Modus völlig aus.
- Eigene Testhashes aus frei erfundenen Passwörtern reichen für alle Übungen aus.

Kapitel 4: Angriffsmodi

Lernziele

- Du kennst die vier wichtigsten Angriffsmodi von Hashcat und ihre Nummerierung.
- Du kannst für eine gegebene Aufgabe den passenden Modus auswählen.
- Du verstehst, wie sich die Modi zueinander verhalten.

4.1 Die Angriffsmodi im Überblick

Anders als John the Ripper (benannte Modi wie „Single Crack") nummeriert Hashcat seine Angriffsmodi durchgehend über die Option `-a`.

Modus	Option	Grundidee
Dictionary	`-a 0`	probiert jeden Eintrag einer Wortliste, optional mit Regeln (Kapitel 6)
Combination	`-a 1`	kombiniert zwei Wortlisten paarweise miteinander
Brute-Force / Maske	`-a 3`	probiert systematisch Zeichenkombinationen nach einer Maske (Kapitel 7)
Hybrid (Wortliste + Maske)	`-a 6` / `-a 7`	hängt eine Maske an jeden Wortlisteneintrag an (`-a 6`) oder davor (`-a 7`)

4.2 Dictionary-Angriff (-a 0)

Beispiel 4.1: Grundlegender Wortlisten-Angriff

```
$ hashcat -a 0 -m 0 testhashes.txt testliste.txt
```

`-m 0` gibt den Hash-Modus an (hier: unsalted MD5, vertieft in Kapitel 5). Dies ist der direkte Gegenpart zum Wortlisten-Modus in John the Ripper und in der Praxis der am häufigsten genutzte Angriffsmodus.

4.3 Combination-Angriff (-a 1)

Beispiel 4.2: Zwei Wortlisten kombinieren

```
$ hashcat -a 1 -m 0 testhashes.txt liste1.txt liste2.txt
```

Beispiel 4.3: Was dabei entsteht (Konzept)

```
liste1.txt: Sommer, Winter
liste2.txt: 2026, 2027
Erzeugte Kandidaten: Sommer2026, Sommer2027, Winter2026, Winter2027
```

Dieser Modus eignet sich, wenn du vermutest, dass Passwörter aus zwei erkennbaren Bausteinen bestehen – etwa einem Wort und einer Jahreszahl.

4.4 Brute-Force- bzw. Maskenangriff (-a 3)

Beispiel 4.4: Maskenbasierter Angriff

```
$ hashcat -a 3 -m 0 testhashes.txt '?u?l?l?l?l?l?d?d'
```

Dieser Modus entspricht dem `--mask`-Parameter in John the Ripper (John-the-Ripper-Handbuch, Kapitel 7.2) und wird in Kapitel 7 dieses Buches vertieft.

4.5 Hybrid-Angriff (-a 6 / -a 7)

Beispiel 4.5: Wortliste mit angehängter Maske kombinieren

```
$ hashcat -a 6 -m 0 testhashes.txt testliste.txt '?d?d'
```

Beispiel 4.6: Was dabei entsteht (Konzept)

```
Wortlisteneintrag: sommer
Maske: ?d?d (zwei beliebige Ziffern)
Erzeugte Kandidaten: sommer00, sommer01, ..., sommer99
```

`-a 7` funktioniert spiegelbildlich: Die Maske wird **vor** den Wortlisteneintrag gesetzt, statt dahinter.

> **TIPP – Profi-Trick**
>
> Der Hybrid-Modus ist oft der beste Kompromiss zwischen der Treffsicherheit einer Wortliste und der Vollständigkeit einer Maske: Er testet ein bekanntes Grundwort systematisch mit allen plausiblen Ergänzungen, ohne den riesigen Suchraum eines vollständigen Brute-Force-Angriffs über denselben Zeichensatz.

Zusammenfassung

> **Zusammenfassung**
>
> - Hashcat nummeriert seine Angriffsmodi durchgehend: `-a 0` Dictionary, `-a 1` Combination, `-a 3` Brute-Force/Maske, `-a 6`/`-a 7` Hybrid.
> - Dictionary ist der am häufigsten genutzte, Combination eignet sich für zwei erkennbare Bausteine.
> - Brute-Force/Maske entspricht dem Maskenkonzept aus dem John-the-Ripper-Handbuch.
> - Hybrid-Angriffe kombinieren die Treffsicherheit einer Wortliste mit der Systematik einer Maske.

Kapitel 5: Hash-Modi erkennen und angeben

Lernziele

- Du findest den passenden Hash-Modus für ein gegebenes Hash-Format.
- Du verstehst, warum die exakte Modusnummer entscheidend ist.
- Du kennst gängige Modusnummern für häufige Formate.

5.1 Warum die exakte Nummer zählt

Jedes von Hashcat unterstützte Hash-Verfahren hat eine eigene, feste **Modusnummer** (`-m`), die über die Option `-m` angegeben wird. Anders als bei John the Ripper, das Formate meist automatisch gut erkennt, verlangt Hashcat grundsätzlich die explizite Angabe.

Beispiel 5.1: Alle unterstützten Modi durchsuchen

```
$ hashcat --help | grep -i "md5"
$ hashcat --example-hashes | less
```

5.2 Häufige Hash-Modi

Modusnummer	Format
0	MD5 (unsalted)
100	SHA-1 (unsalted)
1400	SHA-256 (unsalted)
1000	NTLM (Windows)
1800	sha512crypt (modernes Unix-Format)
3200	bcrypt
22000	WPA-PBKDF2-PMKID+EAPOL (moderne WLAN-Handshakes)

Beispiel 5.2: Modus explizit angeben

```
$ hashcat -a 0 -m 1000 ntlm_hashes.txt testliste.txt
```

WARNUNG – Sicherheitsrisiko

Ein falsch angegebener Modus führt nicht zu einer Fehlermeldung über „falsches Format", sondern meist einfach dazu, dass **kein einziger Treffer** gefunden wird, selbst wenn das richtige Passwort in deiner Wortliste steht – Hashcat berechnet dann schlicht nach dem falschen Verfahren. Prüfe die Modusnummer bei unerwartet erfolglosen Läufen immer zuerst.

5.3 Beispiel-Hashes zum Nachschlagen

Beispiel 5.3: Ein Beispiel für einen bestimmten Modus anzeigen

```
$ hashcat --example-hashes -m 1000
Hash-Mode....: 1000
Hash-Name....: NTLM
Example.Hash.: b4b9b02e6f09a9bd760f388b67351e2b
```

TIPP – Profi-Trick

`--example-hashes` zeigt dir für jeden Modus ein Beispiel im korrekten Format. Ein Vergleich zwischen deinem eigenen Hash und diesem Beispiel ist oft die schnellste Methode, um eine falsche Modusvermutung zu erkennen, bevor du überhaupt einen Lauf startest.

5.4 Testhashes mit passendem Modus erzeugen

Beispiel 5.4: Verschiedene Testhashes für die Übungen erzeugen

```
$ echo -n "testpass123" | md5sum             # passt zu -m 0
$ echo -n "testpass123" | sha256sum          # passt zu -m 1400
```

Zusammenfassung

Zusammenfassung

- Hashcat verlangt grundsätzlich die explizite Angabe der Modusnummer über `-m`.
- Häufige Modi sind u. a. `0` (MD5), `1000` (NTLM), `1400` (SHA-256) und `3200` (bcrypt).
- Ein falscher Modus führt meist zu stillem Misserfolg, nicht zu einer eindeutigen Fehlermeldung.
- `--example-hashes` zeigt für jeden Modus ein Vergleichsbeispiel im korrekten Format.

Kapitel 6: Regeln in Hashcat

Lernziele

- Du wendest vordefinierte Regeldateien auf einen Dictionary-Angriff an.
- Du verstehst die Grundsyntax eigener Hashcat-Regeln.
- Du kannst mehrere Regeldateien kombinieren.

6.1 Dasselbe Prinzip wie bei John the Ripper, andere Syntax

Das Grundprinzip entspricht dem aus dem John-the-Ripper-Handbuch (Kapitel 6.1): Regeln wandeln jeden Wortlisteneintrag automatisch in plausible Varianten um, statt dass die Wortliste jede Variante enthalten müsste.

Beispiel 6.1: Mit einer mitgelieferten Regeldatei arbeiten

```
$ hashcat -a 0 -m 0 testhashes.txt testliste.txt -r rules/best64.rule
```

Die Jumbo-/Standardinstallation bringt bereits mehrere bewährte Regeldateien mit, darunter `best64.rule` – denselben kompakten, wirkungsvollen Regelsatz, der auch im John-the-Ripper-Handbuch (Kapitel 6.2) erwähnt wird.

6.2 Regelsyntax in Grundzügen

Funktion	Zeichen	Beispiel	Wirkung
nichts tun	`:`	`:`	Originalwort unverändert mit ausprobieren
ersten Buchstaben groß	`c`	`c`	`passwort` → `Passwort`
Zeichen anhängen	`$X`	`$1`	`passwort` → `passwort1`
Zeichen voranstellen	`^X`	`^!`	`passwort` → `!passwort`
Zeichen ersetzen	`sXY`	`so0`	ersetzt jedes `o` durch `0`
rückwärts	`r`	`r`	`passwort` → `trowssap`

Beispiel 6.2: Eine eigene, einfache Regeldatei anlegen

```
$ cat > eigene.rule << 'EOF'
c $1
c $1 $2 $3
so0 si1
EOF
$ hashcat -a 0 -m 0 testhashes.txt testliste.txt -r eigene.rule
```

Gut zu wissen

Jede Zeile einer Regeldatei beschreibt **eine eigenständige** Transformation, die auf jeden Wortlisteneintrag angewendet wird. Eine Datei mit 50 Regelzeilen erzeugt also bis zu 50-mal so viele Kandidaten wie die Wortliste selbst – das multipliziert sich mit der Größe der Ausgangsliste schnell zu einer erheblichen Kandidatenmenge.

6.3 Mehrere Regeldateien kombinieren

Beispiel 6.3: Zwei Regeldateien hintereinander anwenden

```
$ hashcat -a 0 -m 0 testhashes.txt testliste.txt -r regeln1.rule -r regeln2.rule
```

Bei zwei Regeldateien wendet Hashcat **jede Regel aus Datei 1 mit jeder Regel aus Datei 2 kombiniert** an – die Anzahl der erzeugten Kandidaten wächst dadurch multiplikativ, nicht additiv.

TIPP – Profi-Trick

Teste eine neue, selbst geschriebene Regeldatei zunächst mit der Debug-Option `--stdout` statt direkt gegen Hashes: `hashcat -a 0 testliste.txt -r eigene.rule --stdout | head` zeigt dir die erzeugten Kandidaten, ohne überhaupt einen Hash zu benötigen.

Zusammenfassung

Zusammenfassung

- Hashcat-Regeln funktionieren wie die aus dem John-the-Ripper-Handbuch bekannten Regeln, mit eigener, kompakter Syntax.
- Mitgelieferte Regeldateien wie `best64.rule` decken bereits viele reale Passwortmuster ab.
- Eigene Regeln lassen sich zeilenweise in einer einfachen Textdatei definieren.
- Mehrere Regeldateien kombiniert vervielfachen die Kandidatenanzahl multiplikativ.

Kapitel 7: Masken und eigene Zeichensätze im Detail

Lernziele

- Du formulierst Masken mit Standard- und eigenen Zeichensätzen.
- Du kannst den Suchraum einer Maske abschätzen und gezielt begrenzen.
- Du setzt inkrementelle Maskenlängen sinnvoll ein.

7.1 Standard-Zeichensatzplatzhalter

Dieselben Grundplatzhalter wie im John-the-Ripper-Handbuch (Kapitel 7.2), mit identischer Bedeutung:

Platzhalter	Bedeutung
`?l`	Kleinbuchstabe (a–z)
`?u`	Großbuchstabe (A–Z)
`?d`	Ziffer (0–9)
`?s`	Sonderzeichen
`?a`	alle druckbaren Zeichen (Kombination aus allen vorherigen)

Beispiel 7.1: Eine Maske mit festem Präfix

```
$ hashcat -a 3 -m 0 testhashes.txt 'Sommer?d?d'
```

7.2 Eigene Zeichensätze

Beispiel 7.2: Einen eigenen Zeichensatz definieren

```
$ hashcat -a 3 -m 0 testhashes.txt -1 '?l?u' '?1?1?1?1?d?d'
```

`-1 '?l?u'` definiert einen **eigenen** Platzhalter `?1`, der hier nur Klein- und Großbuchstaben umfasst (statt aller vier Standardkategorien). Bis zu vier eigene Zeichensätze (`-1` bis `-4`) lassen sich gleichzeitig definieren – nützlich, wenn du ein Passwortmuster kennst, das von den Standardkategorien abweicht (z. B. nur Hexadezimalzeichen).

Beispiel 7.3: Nur Hexadezimalzeichen als eigener Zeichensatz

```
$ hashcat -a 3 -m 0 testhashes.txt -1 '0123456789abcdef' '?1?1?1?1?1?1'
```

7.3 Suchraum abschätzen und begrenzen

Beispiel 7.4: Hashcat schätzt den Suchraum selbst

```
$ hashcat -a 3 -m 0 --keyspace '?u?l?l?l?l?l?d?d'
228826214400
```

Die Option `--keyspace` gibt die Gesamtzahl der Kandidaten aus, **ohne** den eigentlichen Angriff zu starten – ideal, um vorab einzuschätzen, ob eine Maske in angemessener Zeit durchlaufen werden kann.

> **TIPP – Profi-Trick**
>
> Kombiniere `--keyspace` mit dem zuvor gemessenen Benchmark-Wert (Kapitel 8.1): Teilst du die Suchraumgröße durch die Hashes pro Sekunde, erhältst du eine grobe Abschätzung der tatsächlichen Laufzeit, bevor du den Lauf überhaupt startest.

7.4 Inkrementelle Maskenlänge

Beispiel 7.5: Alle Längen von 4 bis 8 Zeichen automatisch durchlaufen

```
$ hashcat -a 3 -m 0 --increment --increment-min=4 --increment-max=8 \
    testhashes.txt '?a?a?a?a?a?a?a?a'
```

`--increment` lässt Hashcat zunächst alle kürzeren Kombinationen probieren, bevor es zur vollen Maskenlänge übergeht – sinnvoll, wenn die tatsächliche Passwortlänge unbekannt ist, aber eine Obergrenze vermutet wird.

Zusammenfassung

> **Zusammenfassung**
>
> - Die Standardplatzhalter `?l`, `?u`, `?d`, `?s`, `?a` entsprechen denen aus dem John-the-Ripper-Handbuch.
> - Eigene Zeichensätze (`-1` bis `-4`) erlauben, Masken auf ein bekanntes, von den Standardkategorien abweichendes Muster zuzuschneiden.
> - `--keyspace` schätzt die Kandidatenanzahl einer Maske, ohne den Angriff zu starten.
> - `--increment` durchläuft automatisch mehrere Längen, von kurz bis zu einer definierten Obergrenze.

Kapitel 8: Performance, Workload-Profile, Sessions und Potfile

Lernziele

- Du misst die tatsächliche Geschwindigkeit deiner Hardware mit dem eingebauten Benchmark.
- Du steuerst die Systemlast während eines Laufs über Workload-Profile.
- Du unterbrichst und setzt lange Läufe gezielt fort.

8.1 Benchmark

Beispiel 8.1: Eingebauten Benchmark ausführen

```
$ hashcat -b
```

Beispiel 8.2: Benchmark für einen bestimmten Modus

```
$ hashcat -b -m 1000    # NTLM
$ hashcat -b -m 3200    # bcrypt
```

Der Unterschied zwischen einem schnellen Modus wie NTLM und einem absichtlich langsamen wie bcrypt beträgt auch hier, wie bereits in Kapitel 2.3 angesprochen, oft mehrere Größenordnungen – unabhängig davon, ob eine GPU zur Verfügung steht.

8.2 Workload-Profile

Beispiel 8.3: Workload-Profil setzen

```
$ hashcat -a 0 -m 0 -w 2 testhashes.txt testliste.txt
```

Profil	Option	Charakteristik
Low	`-w 1`	geringste Systemlast, System bleibt gut nutzbar
Default	`-w 2`	Standardeinstellung
High	`-w 3`	höhere Last, schneller
Nightmare	`-w 4`	maximale Last, System währenddessen kaum nutzbar

TIPP – Profi-Trick

Arbeitest du während eines Laufs noch am selben Rechner weiter (z. B. in einer VM mit gemeinsam genutzten Ressourcen), ist `-w 1` oder `-w 2` die verträglichere Wahl. Für einen dedizierten Lauf über Nacht spricht nichts gegen `-w 3` oder `-w 4`.

8.3 Sessions: lange Läufe unterbrechen und fortsetzen

Beispiel 8.4: Einen benannten Lauf starten

```
$ hashcat -a 0 -m 0 --session=testlauf testhashes.txt testliste.txt -r best64.rule
```

Mit `Strg+C` unterbrichst du einen Lauf, ohne den Fortschritt zu verlieren.

Beispiel 8.5: Einen unterbrochenen Lauf fortsetzen

```
$ hashcat --session=testlauf --restore
```

Beispiel 8.6: Status während des Laufs abfragen

```
# Während des laufenden Prozesses:
s    # Status anzeigen
p    # pausieren
r    # fortsetzen
q    # beenden (Fortschritt bleibt für --restore erhalten)
```

8.4 Die Potfile

Wie bei John the Ripper landet jedes gefundene Passwort automatisch in einer **Potfile** (standardmäßig `hashcat.potfile` im Hashcat-Konfigurationsverzeichnis).

Beispiel 8.7: Bereits gefundene Passwörter anzeigen

```
$ hashcat -m 0 testhashes.txt --show
```

WARNUNG – Sicherheitsrisiko

Wie die Potfile von John the Ripper sammelt auch die Hashcat-Potfile über die Zeit alle jemals geknackten Passwörter an. Sie kann reale, sensible Zugangsdaten im Klartext enthalten und gehört entsprechend geschützt: verschlüsselt speichern, nicht unnötig lange aufbewahren, nicht ungeschützt weitergeben (Ethical-Hacking-Handbuch, Kapitel 8.5).

Zusammenfassung

Zusammenfassung

- `hashcat -b` misst die tatsächliche Geschwindigkeit deiner Hardware je Hash-Modus.
- Workload-Profile (`-w 1` bis `-w 4`) steuern den Kompromiss zwischen Systemlast und Geschwindigkeit.
- `--session` und `--restore` erlauben, lange Läufe gezielt zu unterbrechen und fortzusetzen.
- Die Potfile sammelt alle gefundenen Passwörter und muss entsprechend sorgfältig behandelt werden.

Kapitel 9: hashcat-utils und ergänzende Werkzeuge

Lernziele

- Du kennst die wichtigsten Zusatzwerkzeuge aus dem hashcat-utils-Projekt.
- Du nutzt sie, um Wortlisten gezielt vorzubereiten und zu kombinieren.
- Du weißt, wann sich ein Zusatzwerkzeug lohnt statt einer eingebauten Hashcat-Option.

9.1 Was ist hashcat-utils?

hashcat-utils ist eine separat gepflegte Sammlung kleiner Kommandozeilenwerkzeuge, die typische Vor- und Nachbereitungsaufgaben rund um Wortlisten übernehmen – Aufgaben, die Hashcat selbst nicht direkt abdeckt.

Beispiel 9.1: hashcat-utils installieren (falls nicht bereits enthalten)

```
$ sudo apt install hashcat-utils
```

9.2 Wortlisten bereinigen und kombinieren

Beispiel 9.2: Doppelte Einträge entfernen

```
$ cat testliste.txt | sort -u > testliste_bereinigt.txt
```

Beispiel 9.3: Zwei Wortlisten mit combinator.bin zusammenführen

```
$ combinator.bin liste1.txt liste2.txt > kombiniert.txt
```

`combinator.bin` erzeugt dasselbe Ergebnis wie der Combination-Angriff `-a 1` (Kapitel 4.3), speichert die Kombinationen aber als eigenständige Wortliste – nützlich, wenn du dieselbe kombinierte Liste später mehrfach oder mit Regeln weiterverwenden möchtest.

9.3 Wörter aus bestehenden Treffern ableiten

Beispiel 9.4: Wortlänge und Zeichensatz einer bestehenden Liste analysieren

```
$ statsgen.bin testliste.txt
```

`statsgen.bin` wertet eine vorhandene Wortliste (z. B. bereits geknackte Passwörter aus Kapitel 10) statistisch aus: Wie lang sind die Einträge typischerweise, welche Zeichenklassen kommen vor? Diese Auswertung liefert wertvolle Hinweise für eine gezielte Maske (Kapitel 7), statt sie zu raten.

Beispiel 9.5: Aus Wörtern automatisch Masken ableiten

```
$ maskgen.bin testliste.txt
```

`maskgen.bin` schlägt passende Masken direkt aus einer Wortliste vor – ein praktischer Einstieg, wenn du von „ich habe bereits einige Passwörter geknackt" zu „welche Maske deckt die übrigen am

wahrscheinlichsten ab" kommen möchtest (vergleiche John-the-Ripper-Handbuch, Kapitel 7.3).

9.4 Wann lohnt sich ein Zusatzwerkzeug?

Aufgabe	Eingebaute Hashcat-Option	hashcat-utils-Alternative
Zwei Listen kombinieren	`-a 1` (nur während des Laufs, nicht gespeichert)	`combinator.bin` (speichert das Ergebnis dauerhaft)
Muster in Treffern erkennen	keine eingebaute Option	`statsgen.bin`, `maskgen.bin`
Wortliste bereinigen	keine eingebaute Option	Standard-Shell-Werkzeuge (`sort -u`)

TIPP – Profi-Trick

hashcat-utils lohnt sich vor allem bei wiederkehrenden Aufgaben oder wenn ein Zwischenergebnis (z. B. eine kombinierte Liste) für mehrere spätere Läufe wiederverwendet werden soll.

Zusammenfassung

Zusammenfassung

- hashcat-utils ergänzt Hashcat um Werkzeuge zur Wortlisten-Vor- und Nachbereitung.
- `combinator.bin` erzeugt dauerhaft gespeicherte Kombinationen, `statsgen.bin` und `maskgen.bin` leiten Muster und Masken aus bestehenden Treffern ab.
- Für einfache Aufgaben wie das Entfernen von Duplikaten reichen oft Standard-Shell-Werkzeuge.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust das für dieses Buch nötige Labor auf.
- Du arbeitest acht Übungen durch, die alle vorherigen Kapitel praktisch verbinden.
- Du weißt, dass die Laufzeit einzelner Übungen stark von deiner Hardware (CPU- oder GPU-Modus) abhängt.

10.1 Laboraufbau

Für dieses Buch genügt eine einzelne Kali-VM mit installiertem Hashcat (Kapitel 3). Ein zweites Zielsystem ist nicht nötig, da ausschließlich mit lokal erzeugten Testhashes gearbeitet wird.

10.2 Checkliste: sicherer Umgang mit Testdaten

Checkliste 10.1: Vor jeder Übung

```
[ ] Alle verwendeten Hashes stammen aus selbst erzeugten, frei erfundenen
    Testpasswörtern (Kapitel 3.3) – keine echten Zugangsdaten
[ ] Der verwendete Hash-Modus (-m) wurde mit --example-hashes gegengeprüft
[ ] Lange Läufe (Brute-Force, große Masken) mit --keyspace vorab abgeschätzt
[ ] Potfile nach Abschluss der Übungen bei Bedarf gelöscht
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und Fehlerbehebung.

Übung 1: Rechengeräte und Benchmark prüfen

Ziel: Die eigene Hardware-Ausstattung und Geschwindigkeit kennenlernen. **Voraussetzungen:** Kali-VM mit Hashcat. **Durchführung:** Führe `hashcat -I` (Beispiel 2.2) und danach `hashcat -b` (Beispiel 8.1) aus. **Erwartetes Ergebnis:** Eine Liste erkannter Rechengeräte sowie Geschwindigkeitswerte für mehrere Hash-Modi. **Auswertung:** Notiere, ob CPU- oder GPU-Geräte erkannt wurden, und die Geschwindigkeit für MD5 als Referenzwert für spätere Übungen. **Fehlerbehebung:** Werden keine GPU-Geräte erkannt, erzwinge mit `-D 1` den CPU-Modus (Beispiel 3.3) und fahre mit den übrigen Übungen fort – alle Konzepte bleiben identisch.

Übung 2: Dictionary-Angriff mit und ohne Regeln

Ziel: Die Wirkung von Regeln auf einen Dictionary-Angriff messen. **Voraussetzungen:** ein Testhash für ein Passwort wie `Sommer2026!`, das auf `sommer` basiert, aber nicht wörtlich in deiner Testliste steht. **Durchführung:** Führe zunächst `hashcat -a 0 -m 0 testhashes.txt testliste.txt` ohne Regeln aus, danach mit `-r best64.rule` (Beispiel 6.1) gegen denselben, zurückgesetzten Hash. **Erwartetes Ergebnis:** Ohne Regeln kein Treffer, mit Regeln ein Treffer. **Auswertung:** Vergleiche mit demselben Versuch aus dem John-the-Ripper-Handbuch (Übung 2 dort) – beide Werkzeuge sollten zum selben inhaltlichen Ergebnis kommen. **Fehlerbehebung:** Setze die Potfile zwischen den beiden Läufen zurück oder nutze einen neuen Hash, sonst meldet Hashcat das Passwort fälschlich sofort aus dem Cache.

Übung 3: Combination-Angriff einsetzen

Ziel: Zwei erkennbare Passwortbausteine gezielt kombinieren. **Voraussetzungen:** ein Testhash für ein Passwort wie `Winter2027`. **Durchführung:** Lege zwei kleine Listen an (`jahreszeiten.txt` mit „Sommer", „Winter"; `jahre.txt` mit „2026", „2027") und führe `hashcat -a 1 -m 0 testhashes.txt jahreszeiten.txt jahre.txt` aus (Beispiel 4.2). **Erwartetes Ergebnis:** Ein Treffer, da die Kombination aus beiden Listen genau das Testpasswort ergibt. **Auswertung:** Zähle die Gesamtzahl der erzeugten Kombinationen (Anzahl Liste 1 × Anzahl Liste 2) und vergleiche sie mit der tatsächlichen Laufzeit. **Fehlerbehebung:** Kein Treffer? Prüfe, ob Groß-/Kleinschreibung in deinen Listen exakt zum Testpasswort passt – Combination-Angriffe verändern die Schreibweise nicht automatisch.

Übung 4: Suchraum einer Maske abschätzen

Ziel: `--keyspace` zur Planung eines Maskenangriffs nutzen. **Voraussetzungen:** wie Übung 1. **Durchführung:** Führe `hashcat -a 3 -m 0 --keyspace '?u?l?l?l?l?l?d?d'` aus (Beispiel 7.4) und danach denselben Angriff ohne `--keyspace` gegen einen passenden Testhash. **Erwartetes Ergebnis:** `--keyspace` liefert eine Zahl im zweistelligen Milliardenbereich; der tatsächliche Lauf dauert entsprechend lange oder findet den Treffer früher, falls er am Anfang der Reihenfolge liegt. **Auswertung:** Teile die Keyspace-Zahl durch deinen Benchmark-Wert aus Übung 1 und vergleiche die Abschätzung mit der real gemessenen Zeit bis zum Treffer. **Fehlerbehebung:** Dauert der Lauf unerwartet lange, brich ihn ab und prüfe, ob die Maske wirklich zur Länge und Struktur deines Testpassworts passt.

Übung 5: Hybrid-Angriff einsetzen

Ziel: Ein bekanntes Grundwort systematisch mit einer Zahlenendung ergänzen. **Voraussetzungen:** ein Testhash für `testpass42`. **Durchführung:** Führe `hashcat -a 6 -m 0 testhashes.txt testliste.txt '?d?d'` aus (Beispiel 4.5), wobei `testliste.txt` den Eintrag `testpass` enthält. **Erwartetes Ergebnis:** Ein Treffer, da der Hybrid-Modus `?d?d` an jeden Wortlisteneintrag anhängt. **Auswertung:** Vergleiche die Laufzeit mit einem äquivalenten reinen Maskenangriff (`?a?a?a?a?a?a?a?a?d?d`) über denselben Zeichenraum – der Hybrid-Angriff sollte deutlich schneller zum Ziel kommen. **Fehlerbehebung:** Kein Treffer? Prüfe, ob `?d?d` tatsächlich zu einer zweistelligen Zahl passt, nicht zu einer ein- oder dreistelligen.

Übung 6: Workload-Profile vergleichen

Ziel: Den Einfluss der Workload-Profile auf Geschwindigkeit und Systemlast beobachten. **Voraussetzungen:** wie Übung 1, idealerweise mit einer Aufgabenüberwachung (z. B. `htop`) parallel geöffnet. **Durchführung:** Führe denselben Dictionary-Angriff mit `-w 1` und danach mit `-w 3` aus (Beispiel 8.3), und beobachte dabei die Systemauslastung. **Erwartetes Ergebnis:** `-w 3` zeigt eine spürbar höhere Auslastung als `-w 1`, bei entsprechend kürzerer Laufzeit für größere Kandidatenmengen. **Auswertung:** Ordne das Verhältnis zwischen Last und Geschwindigkeit dem Zielkonflikt aus Kapitel 8.2 zu. **Fehlerbehebung:** Bei sehr kleinen Wortlisten ist der Unterschied kaum messbar, da der Lauf ohnehin fast sofort beendet ist – wiederhole die Übung mit einer deutlich größeren Testliste.

Übung 7: Session unterbrechen und fortsetzen

Ziel: Session-Verwaltung praktisch nutzen. **Voraussetzungen:** ein zeitlich länger laufender Angriff (z. B. ein Brute-Force mit mehreren Stellen). **Durchführung:** Starte `hashcat -a 3 -m 0 --session=testlauf '?a?a?a?a?a?a'` gegen einen Testhash, unterbrich nach kurzer Zeit mit `q` (Beispiel 8.6), setze danach mit `hashcat --session=testlauf --restore` fort. **Erwartetes Ergebnis:** Der Lauf setzt dort fort, wo er unterbrochen wurde. **Auswertung:** Vergleiche die kombinierte Laufzeit beider Teile mit einem ununterbrochenen Lauf gleicher Gesamtdauer. **Fehlerbehebung:** Meldet `--restore` einen Fehler, prüfe, ob der exakte Sitzungsname aus `--session` verwendet wurde.

Übung 8: hashcat-utils einsetzen

Ziel: Aus vorhandenen Treffern eine gezielte Maske ableiten. **Voraussetzungen:** mehrere bereits geknackte Testpasswörter aus den vorherigen Übungen, hashcat-utils installiert. **Durchführung:** Sammle die gefundenen Klartext-Passwörter in einer Datei und führe `statsgen.bin` sowie `maskgen.bin` darauf aus (Beispiel 9.4–9.5). **Erwartetes Ergebnis:** Eine statistische Übersicht sowie vorgeschlagene Masken, die zu den Mustern deiner Testpasswörter passen. **Auswertung:** Vergleiche die vorgeschlagenen Masken mit den Mustern, die du in den vorherigen Übungen selbst gewählt hast. **Fehlerbehebung:** Sind die Werkzeuge nicht installiert, prüfe mit `which combinator.bin`, ob das Paket `hashcat-utils` tatsächlich installiert wurde (Beispiel 9.1).

Zusammenfassung

Zusammenfassung

- Für dieses Buch genügt eine einzelne Kali-VM; alle Übungen arbeiten mit selbst erzeugten Testhashes.
- Die Übungen verbinden alle vier Angriffsmodi, Regeln, Masken, Performance-Steuerung und hashcat-utils.
- Mehrere Übungen verweisen bewusst auf das John-the-Ripper-Handbuch, um dieselbe Fragestellung mit beiden Werkzeugen zu vergleichen.

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst die häufigsten Hashcat-Probleme einordnen und beheben.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Probleme

Meldung / Symptom	Wahrscheinliche Ursache	Lösung
„No devices found/left"	fehlende oder veraltete GPU-Treiber	Treiber aktualisieren, mit `-D 1` auf CPU-Modus ausweichen (Kapitel 3.2)
Lauf findet trotz richtigem Kandidaten nichts	falscher Hash-Modus (`-m`) angegeben	mit `--example-hashes -m <nummer>` gegenprüfen (Kapitel 5.3)
„Already cracked" ohne erkennbaren Grund	Treffer steckt bereits in der Potfile	`--show` statt erneutem Lauf nutzen, oder Potfile gezielt zurücksetzen
Brute-Force-Lauf wirkt endlos	zu großer Suchraum ohne vorherige Abschätzung	`--keyspace` vor dem Start prüfen (Kapitel 7.3), Maske oder Hybrid-Modus erwägen
`--restore` schlägt fehl	Sitzungsname falsch oder Sitzung bereits abgeschlossen	exakten `--session`-Namen verwenden
System wird während des Laufs unbenutzbar	zu hohes Workload-Profil	`-w 1` oder `-w 2` statt `-w 4` setzen (Kapitel 8.2)

11.2 Spickzettel

Beispiel 11.1: Die wichtigsten Hashcat-Befehle auf einen Blick

```
# Rechengeräte und Geschwindigkeit prüfen
hashcat -I
hashcat -b

# Dictionary-, Combination-, Brute-Force- und Hybrid-Angriff
hashcat -a 0 -m <modus> hashes.txt liste.txt
hashcat -a 1 -m <modus> hashes.txt liste1.txt liste2.txt
hashcat -a 3 -m <modus> hashes.txt '?u?l?l?l?d?d'
hashcat -a 6 -m <modus> hashes.txt liste.txt '?d?d'

# Mit Regeln kombinieren
hashcat -a 0 -m <modus> hashes.txt liste.txt -r best64.rule

# Suchraum abschätzen, ohne zu starten
hashcat -a 3 -m <modus> --keyspace '?a?a?a?a?a?a'
# Ergebnisse ansehen
hashcat -m <modus> hashes.txt --show

# Session-Verwaltung
hashcat --session=lauf1 -a 0 -m <modus> hashes.txt liste.txt
hashcat --session=lauf1 --restore

# Beispiel für einen Hash-Modus anzeigen
hashcat --example-hashes -m <modus>
```

11.3 Glossar

- **Combination-Angriff (-a 1)** – kombiniert zwei Wortlisten paarweise miteinander.
- **CUDA** – NVIDIA-spezifische Schnittstelle zur GPU-Programmierung.
- **Dictionary-Angriff (-a 0)** – probiert jeden Eintrag einer Wortliste.
- **Hash-Modus (-m)** – die feste Kennnummer eines bestimmten Hash-Verfahrens in Hashcat.
- **hashcat-utils** – separat gepflegte Zusatzwerkzeuge zur Wortlisten-Vor- und Nachbereitung.
- **Hybrid-Angriff (-a 6/-a 7)** – kombiniert eine Wortliste mit einer angehängten oder vorangestellten Maske.
- **Keyspace** – die Gesamtzahl der Kandidaten eines Angriffs, mit `--keyspace` abschätzbar.
- **OpenCL** – herstellerunabhängige Schnittstelle zur GPU-Programmierung.
- **Potfile** – die Datei, in der Hashcat alle jemals gefundenen Passwörter sammelt.
- **Workload-Profil (-w)** – steuert den Kompromiss zwischen Systemlast und Cracking-Geschwindigkeit.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
Offizielle Hashcat-Dokumentation (hashcat.net/wiki)	vollständige, verbindliche Referenz inklusive aller Hash-Modi
John-the-Ripper-Handbuch (dieselbe Buchreihe)	CPU-zentriertes Gegenstück mit identischen Grundkonzepten
Ethical-Hacking-Handbuch (dieselbe Buchreihe)	Gesamtzusammenhang: Online- vs. Offline-Passwortangriffe, Reporting
hashcat-utils-Projektseite	Dokumentation aller Zusatzwerkzeuge aus Kapitel 9
TryHackMe / HackTheBox	legale Übungsplattformen mit eigenen Passwort-Cracking-Räumen

TIPP – Profi-Trick

Das Hashcat-Wiki listet für jeden unterstützten Hash-Modus die genaue Modusnummer, ein Beispiel-Hash und oft zusätzliche Hinweise zur korrekten Formatierung – bei einem unbekannten Hash-Format ist das meist die schnellste Anlaufstelle.

Zusammenfassung

Zusammenfassung

- Die häufigsten Probleme betreffen fehlende GPU-Treiber, falsche Hash-Modi und unbegrenzte Brute-Force-Läufe.
- Der Spickzettel deckt die wichtigsten Befehlsmuster aller vorherigen Kapitel ab.
- Die offizielle Dokumentation, das John-the-Ripper- und das Ethical-Hacking-Handbuch sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Typischer Ablauf eines Hashcat-Laufs (Konzept) … Seite 8
Beispiel 2.1: Grobe Größenordnung (stark vereinfacht, Konzept) … Seite 9
Beispiel 2.2: Verfügbare Rechengeräte auflisten … Seite 9
Beispiel 2.3: Relativer GPU-Vorteil nach Hash-Verfahren (Konzept, keine exakten Werte) … Seite 10
Beispiel 3.1: Hashcat installieren … Seite 11
Beispiel 3.2: Prüfen, welche Rechengeräte Hashcat erkennt … Seite 11
Beispiel 3.3: Hashcat im reinen CPU-Modus erzwingen … Seite 11
Beispiel 3.4: Testhashes aus frei erfundenen Passwörtern erzeugen … Seite 12
Beispiel 4.1: Grundlegender Wortlisten-Angriff … Seite 13
Beispiel 4.2: Zwei Wortlisten kombinieren … Seite 13
Beispiel 4.3: Was dabei entsteht (Konzept) … Seite 13
Beispiel 4.4: Maskenbasierter Angriff … Seite 14
Beispiel 4.5: Wortliste mit angehängter Maske kombinieren … Seite 14
Beispiel 4.6: Was dabei entsteht (Konzept) … Seite 14
Beispiel 5.1: Alle unterstützten Modi durchsuchen … Seite 15
Beispiel 5.2: Modus explizit angeben … Seite 15
Beispiel 5.3: Ein Beispiel für einen bestimmten Modus anzeigen … Seite 16
Beispiel 5.4: Verschiedene Testhashes für die Übungen erzeugen … Seite 16
Beispiel 6.1: Mit einer mitgelieferten Regeldatei arbeiten … Seite 17
Beispiel 6.2: Eine eigene, einfache Regeldatei anlegen … Seite 17
Beispiel 6.3: Zwei Regeldateien hintereinander anwenden … Seite 18
Beispiel 7.1: Eine Maske mit festem Präfix … Seite 19
Beispiel 7.2: Einen eigenen Zeichensatz definieren … Seite 19
Beispiel 7.3: Nur Hexadezimalzeichen als eigener Zeichensatz … Seite 19
Beispiel 7.4: Hashcat schätzt den Suchraum selbst … Seite 19
Beispiel 7.5: Alle Längen von 4 bis 8 Zeichen automatisch durchlaufen … Seite 20
Beispiel 8.1: Eingebauten Benchmark ausführen … Seite 21
Beispiel 8.2: Benchmark für einen bestimmten Modus … Seite 21
Beispiel 8.3: Workload-Profil setzen … Seite 21
Beispiel 8.4: Einen benannten Lauf starten … Seite 22
Beispiel 8.5: Einen unterbrochenen Lauf fortsetzen … Seite 22
Beispiel 8.6: Status während des Laufs abfragen … Seite 22
Beispiel 8.7: Bereits gefundene Passwörter anzeigen … Seite 22
Beispiel 9.1: hashcat-utils installieren (falls nicht bereits enthalten) … Seite 23
Beispiel 9.2: Doppelte Einträge entfernen … Seite 23
Beispiel 9.3: Zwei Wortlisten mit combinator.bin zusammenführen … Seite 23
Beispiel 9.4: Wortlänge und Zeichensatz einer bestehenden Liste analysieren … Seite 23
Beispiel 9.5: Aus Wörtern automatisch Masken ableiten … Seite 23
Checkliste 10.1: Vor jeder Übung … Seite 25
Beispiel 11.1: Die wichtigsten Hashcat-Befehle auf einen Blick … Seite 29

Index

best64.rule – 17, 26
Brute-Force – 14, 27
Combination-Angriff – 13, 23, 26, 29
CPU – 5, 7, 9, 10, 11, 25
CUDA – 9, 29
Dictionary-Angriff – 13, 26, 27, 29
GPU – 5, 7, 9, 10, 11, 21, 25, 29
Hash-Modus – 13, 29
Hashcat – 5, 7, 8, 9, 11, 12, 13, 15
hashcat-utils – 23, 27, 29
Hybrid-Angriff – 14, 26, 29
John the Ripper – 5, 7, 13, 14, 15, 17, 22
Keyspace – 20, 26, 29
Maske – 5, 14, 19, 20, 23, 26, 27, 29
OpenCL – 9, 29
Potfile – 22, 26, 29
Regel – 5, 17, 18, 23, 26
Session – 22, 27
Workload-Profil – 5, 21, 27, 29

HASHCAT DAS PRAXISHANDBUCH

Wenn tausend Kerne gleichzeitig raten

Hashcat löst dieselbe Aufgabe wie John the Ripper aus dieser Reihe – Passwort-Hashes offline prüfen –, nutzt dafür aber von Grund auf die massive Parallelität moderner Grafikkarten. Dieses Handbuch führt durch Dictionary-, Combination-, Brute-Force- und Hybrid-Angriffe, eigene Zeichensätze und Masken, Benchmarks und Workload-Profile, und zeigt durchgehend, wo sich Hashcat von John the Ripper unterscheidet und wo beide dieselbe Sprache sprechen.

DAS ERWARTET DICH

- GPU- statt CPU-zentriertes Cracking: OpenCL, CUDA und der reale Geschwindigkeitsunterschied
- Vier Angriffsmodi: Dictionary, Combination, Brute-Force/Maske und Hybrid
- Eigene Zeichensätze, Keyspace-Abschätzung und inkrementelle Maskenlängen
- Benchmarks, Workload-Profile, Sessions und Potfile-Verwaltung
- hashcat-utils, 8 geführte Übungen mit selbst erzeugten Testhashes, Spickzettel und Glossar

Für wen?

Leser:innen des John-the-Ripper- und Ethical-Hacking-Handbuchs, die das GPU-zentrierte Gegenstück kennenlernen möchten, sowie Studierende und Auszubildende. Ausschließlich für selbst erzeugte Testhashes und autorisierte Audits.

32 Seiten · Stand Oktober 2026 · Ergänzung zum Ethical-Hacking- und John-the-Ripper-Handbuch

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.